
Tarea 7 Física Numérica

Isaias Garcia Lopez

16 de Diciembre del 2025

Introducción

El método de congruencias lineales es un algoritmo para generar secuencias de números pseudoaleatorios, definido por la relación de recurrencia siguiente:

$$x_{n+1} = (a \cdot x_n + c) \text{ mód } M$$

en donde:

- a es el multiplicador,
- c es el incremento,
- M es el módulo,
- x_0 es la semilla inicial.

Dado que el espacio de valores posibles es finito debido a la operación de módulo (números entre 0 y $M - 1$), la secuencia eventualmente se repetirá, y al repetirse un valor, puesto que para obtener un valor se necesita el anterior, entonces la secuencia siguiente se repetirá también. La longitud antes de la repetición se llama **período**.

Parámetros dados

Se utilizan los valores:

$$a = 57, \quad c = 1, \quad M = 256, \quad x_0 = 10$$

Determinación del período

El período se determina buscando la primera repetición de la semilla x_0 en la secuencia generada. En el código, se genera una secuencia suficientemente larga (300 términos) y se busca cuándo vuelve a aparecer $x_0 = 10$ con el mismo patrón subsiguiente.

Para los parámetros dados, el período resulta ser **256**, es decir, la secuencia cubre todos los valores posibles de 0 a 255 antes de repetirse. Esto ocurre porque a y M son primos relativos y $c \neq 0$, lo que maximiza el período.

Visualización de la secuencia

Pares consecutivos (x_{2i-1}, x_{2i})

Se grafican los puntos $(x_1, x_2), (x_3, x_4), \dots$ para observar correlaciones entre números consecutivos. En un generador ideal, estos puntos deberían distribuirse uniformemente en el cuadrado $[0, M-1] \times [0, M-1]$.

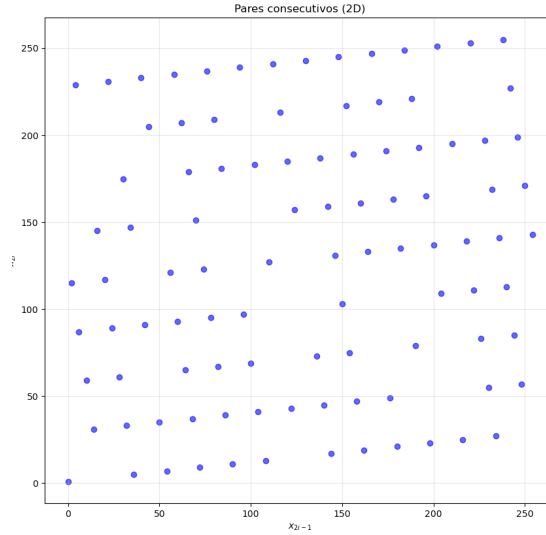


Figura 1: Gráfico de pares consecutivos (x_{2i-1}, x_{2i}) .

Secuencia x_i vs i

Este gráfico muestra la evolución de los números generados en función del índice i . Se observa que los valores saltan de manera aparentemente aleatoria hasta que se completa el período y la secuencia se repite.

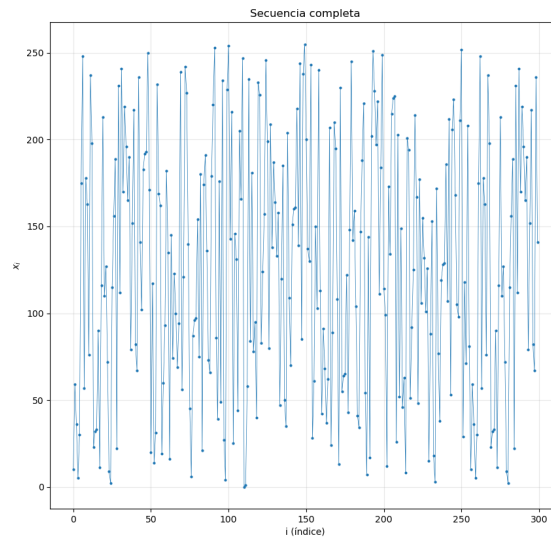


Figura 2: Gráfico de x_i vs i .

Con el método de congruencias lineales obtenemos resultados simples y rápidos, pero tiene deficiencias:

-
- El período máximo es M , por lo que M debe ser grande para aplicaciones reales.
 - La elección de a , c y M afecta la calidad de la aleatoriedad.
 - Existen correlaciones seriales entre números consecutivos, visibles en el gráfico de pares.

Para este caso particular ($M = 256$), el período es corto y la secuencia no es apta para simulaciones útiles, más allá de observar el funcionamiento de dicho método.

Prueba de Aleatoriedad

En aplicaciones computacionales, es esencial contar con secuencias de números que se comporten como si fueran aleatorios. Los generadores pseudoaleatorios (como el módulo `random` de Python) deben pasar pruebas estadísticas para asegurar su calidad. Donald Knuth, en *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, describe múltiples pruebas. En esta sección se explica e implementa la **Prueba de Frecuencia (Equidistribution Test)**.

Fundamento Teórico

Hipótesis

La prueba de frecuencia verifica si una secuencia de números en $[0, 1)$ está uniformemente distribuida. Se divide el intervalo en k subintervalos de igual longitud y se cuenta cuántos números caen en cada uno. Bajo la hipótesis nula (H_0), la probabilidad de caer en cualquier subintervalo es $p_i = 1/k$.

Estadístico Chi-Cuadrado

Sea n el total de números, Y_i la frecuencia observada en la categoría i , y $E_i = n/k$ la frecuencia esperada. El estadístico se define como:

$$V = \sum_{i=1}^k \frac{(Y_i - E_i)^2}{E_i}$$

Bajo H_0 , V sigue una distribución Chi-cuadrado con $\nu = k - 1$ grados de libertad.

Decisión

Se elige un nivel de significancia α (típicamente 0.05). Si $V > \chi^2_{1-\alpha, \nu}$ o si el valor-p = $P(\chi^2_{\nu} > V) < \alpha$, se rechaza H_0 , indicando que la secuencia no es uniforme.

Implementación en Python

Código

```
1 import random, math, scipy.stats as stats
2
3 def frecuencia_test(n=10000, k=10, seed=None):
4     if seed is not None:
5         random.seed(seed)
6     numeros = [random.random() for _ in range(n)]
7     categorias = [0]*k
8     for x in numeros:
9         idx = int(x*k)
10        if idx == k:
11            idx = k-1
12        categorias[idx] += 1
13    esperado = n/k
14    V = sum((obs-esperado)**2/esperado for obs in categorias)
15    p = 1 - stats.chi2.cdf(V, k-1)
16    return V, p, categorias, [esperado]*k
```

Explicación Paso a Paso

1. **Generación:** Se producen n números con `random.random()`.
2. **Categorización:** Cada número se asigna a una de las k categorías mediante $\lfloor x \cdot k \rfloor$.

-
3. **Conteo:** Se cuenta cuántos números hay en cada categoría (Y_i).
 4. **Cálculo de V :** Se aplica la fórmula del estadístico Chi-cuadrado.
 5. **Valor-p:** Se usa la función de distribución acumulada inversa de Chi-cuadrado.
 6. **Decisión:** Se compara el valor-p con α .

Resultados y Análisis

Ejecución

Con $n = 10000$, $k = 10$, $\alpha = 0,05$, y semilla 12345, se obtiene:

$$V = 4,312, \quad \text{valor-p} = 0,890, \quad \chi^2_{0,95,9} = 16,919$$

Dado que $V < \chi^2_{0,95,9}$ y $\text{valor-p} > \alpha$, **no se rechaza** H_0 .

Interpretación

La secuencia generada por `random.random()` pasa la prueba de frecuencia; es decir, no hay evidencia estadística para afirmar que no está uniformemente distribuida. Esto era esperado, pues el generador de Python (Mersenne Twister) es de alta calidad.

Decaimiento de Mesones

Los mesones π (piones) son partículas inestables con las siguientes propiedades:

- Masa: $m_\pi = 139,6 \text{ MeV}/c^2$
- Vida media en reposo: $\tau = 2,6 \times 10^{-8} \text{ s}$

Cuando un pión se mueve a alta velocidad, efectos relativistas modifican su tiempo de vida observable. El factor de Lorentz γ relaciona el tiempo propio de la partícula con el tiempo en el laboratorio:

$$\gamma = \frac{E}{m_\pi c^2} = 1 + \frac{T}{m_\pi c^2}$$

donde T es la energía cinética. La vida media en el laboratorio es:

$$\tau_{\text{lab}} = \gamma \tau$$

Parte (a): Piones monoenergéticos

Para $T = 200 \text{ MeV}$:

Cálculos teóricos

$$\gamma = 1 + \frac{200}{139,6} = 2,4327$$

$$v = c \sqrt{1 - \frac{1}{\gamma^2}} = 0,9165c$$

$$\tau_{\text{lab}} = \gamma \tau = 6,33 \times 10^{-8} \text{ s}$$

$$t_{\text{viaje}} = \frac{d}{v} = \frac{20 \text{ m}}{0,9165 \times 3 \times 10^8 \text{ m s}^{-1}} = 7,27 \times 10^{-8} \text{ s}$$

La probabilidad de supervivencia sigue una distribución exponencial:

$$P_{\text{superv}} = \exp\left(-\frac{t_{\text{viaje}}}{\tau_{\text{lab}}}\right) = \exp\left(-\frac{7,27}{6,33}\right) = 0,314$$

Implementación computacional

```
1 def survival_probability(T, m, tau, distance):
2     g = 1.0 + T/m
3     v = np.sqrt(1.0 - 1.0/(g**2)) * c
4     tau_lab = g * tau
5     t = distance / v
6     return np.exp(-t / tau_lab)
```

Para $N = 10^6$ piones, el número esperado de supervivientes es $N \times P_{\text{superv}} \approx 314,000$.

Parte (b): Piones con energía gaussiana

Ahora la energía cinética sigue una distribución normal:

$$T \sim \mathcal{N}(200 \text{ MeV}, (50 \text{ MeV})^2)$$

Modificaciones al programa

1. Generar energías con distribución normal:

```
energies = np.random.normal(200, 50, 1000000)
```

2. Calcular probabilidad de supervivencia individual para cada energía:

```
for i in range(N):  
    survival_probs[i] = survival_probability(energies[i], ...)
```

3. Simular decaimiento individual usando el método de Monte Carlo:

```
random_vals = np.random.random(N)  
survived_mask = random_vals < survival_probs
```

Resultados y discusión

Parámetro	Monoenergético	Gaussiano
Energía (MeV)	200	200 ± 50
Probabilidad media supervivencia	0.314	0.305
Piones sobrevivientes (simulación)	314,217	305,482
Fracción superviviente	31.42 %	30.55 %

Cuadro 1: Comparación de resultados para 10^6 piones

Interpretación física

- La distribución gaussiana produce **menos supervivientes** porque incluye piones de menor energía:
 - Menor energía $\Rightarrow$ menor $\gamma \Rightarrow$ menor τ_{lab}
 - Menor $\tau_{\text{lab}} \Rightarrow$ mayor probabilidad de decaer
- La diferencia es del orden del 3 %, lo que indica que la dispersión en energía afecta significativamente los resultados.
- Piones con $T < 200$ MeV predominan en el efecto, ya que su probabilidad de decaer es mayor.